

Dynamic Programming

Dmitry Mukhin

d.mukhin@lse.ac.uk

The previous lecture shows that dynamic optimization problems can be much more complex than the static ones because of a large number of periods and states. Such problems, however, also have one important advantage that allows to solve them in a completely different way than the static problems: the variables are naturally ordered by time. As a result, one can use a backward induction and apply the recursive methods.

Recursive approach Consider again a finite-period deterministic savings problem:

$$\begin{aligned} \max_{\{C_t, B_{t+1}\}} \quad & \sum_{t=0}^T \beta^t u(C_t) \\ \text{s.t.} \quad & B_{t+1} = RB_t + Y_t - C_t \\ & B_{T+1} \geq 0. \end{aligned}$$

Following a standard sequential approach, we get the Euler equations

$$u'(C_t) = \beta R u'(C_{t+1}), \quad \forall t < T$$

and the transversality condition

$$B_{T+1} = 0.$$

These optimality conditions together with the budget constraint constitute a system of $2 \times (T + 1)$ equations with $2 \times (T + 1)$ unknowns. As the horizon T goes up, the dimensionality of the problem and the computational burden increase as well.

The key idea of the recursive methods is to solve the problem sequentially, period by period, starting with the very last one and then moving backwards to period zero. In this case, there is no need to solve a large system of equations. Instead, the problem reduces to multiple steps, each computationally very simple. To see this, we solve the model backwards assuming for simplicity log utility $u(C) = \log C$ and zero income $Y_t = 0$:

1. Period T : the optimal decision is trivial as the agent consumes all of her wealth leaving

zero bequest

$$B_{T+1} = 0, \quad C_T = RB_T$$

It follows the utility the household gets in the last period is equal

$$V_T = \log(RB_T)$$

2. Period $T - 1$: the agent starts the period with wealth RB_{T-1} and chooses saving to maximize utility in the two remaining periods of life:

$$\max_{B_T} \left\{ \log(C_{T-1}) + \beta V_T(B_T) \right\}$$

$$\text{s.t. } B_T = RB_{T-1} - C_{T-1}$$

Substitute V_T and the budget constraint into the objective function:

$$\max_{B_T} \left\{ \log(RB_{T-1} - B_T) + \beta \log(RB_T) \right\}$$

The first-order condition

$$\frac{1}{RB_{T-1} - B_T} = \frac{\beta}{B_T}$$

implies

$$B_T = \frac{\beta}{1 + \beta} RB_{T-1}, \quad C_{T-1} = \frac{1}{1 + \beta} RB_{T-1}.$$

and the total utility from the last two periods is equal

$$\begin{aligned} V_{T-1} &= \log C_{T-1} + \beta V_T = \log \frac{1}{1 + \beta} RB_{T-1} + \beta \log \frac{\beta R}{1 + \beta} RB_{T-1} \\ &= (1 + \beta) \log \frac{RB_{T-1}}{1 + \beta} + \beta \log(\beta R). \end{aligned}$$

3. Period $T - 2$: the agent starts the period with wealth RB_{T-2} and chooses saving to maximize utility in the current and all remaining periods:

$$\max_{B_{T-1}} \left\{ \log(C_{T-2}) + \beta V_{T-1}(B_{T-1}) \right\}$$

$$\text{s.t. } B_{T-1} = RB_{T-2} - C_{T-2}$$

Substitute V_{T-1} and the budget constraint into the objective function:

$$\max_{B_{T-1}} \left\{ \log(RB_{T-2} - B_{T-1}) + \beta(1 + \beta) \log \frac{RB_{T-1}}{1 + \beta} + \beta^2 \log(\beta R) \right\}$$

Take the first-order condition

$$\frac{1}{RB_{T-2} - B_{T-1}} = \frac{\beta(1 + \beta)}{B_{T-1}}$$

and solve it to obtain

$$B_{T-1} = \frac{\beta(1 + \beta)}{1 + \beta(1 + \beta)} RB_{T-2}, \quad C_{T-2} = \frac{1}{1 + \beta(1 + \beta)} RB_{T-2}.$$

It follows that the total welfare from the last three periods is equal

$$V_{T-2} = \log C_{T-2} + \beta V_{T-1} = (1 + \beta + \beta^2) \log \frac{RB_{T-2}}{1 + \beta + \beta^2} + \beta(1 + 2\beta) \log(\beta R)$$

4. ...

5. Period t : although we can follow the same steps up until period zero, this problem is simple enough to solve it using the “guess and verify” method. Indeed, the only thing we need to know about V_{t+1} to solve the optimization problem in period t is how it depends on B_{t+1} . The previous steps suggest that

$$V_{t+1} = (1 + \beta + \dots + \beta^{T-t-1}) \log B_{t+1} + \text{const}_{t+1},$$

where const_{t+1} is some constant that depends only on parameters β and R . Using this “educated guess”, the optimization problem of period t can be written as

$$\max_{B_{t+1}} \left\{ \log(RB_t - B_{t+1}) + \beta(1 + \beta + \dots + \beta^{T-t-1}) \log B_{t+1} + \beta \text{const}_{t+1} \right\}.$$

Take the first-order condition

$$\frac{1}{RB_t - B_{t+1}} = \frac{\beta(1 + \beta + \dots + \beta^{T-t-1})}{B_{t+1}}$$

and solve it to get

$$B_{t+1} = \frac{\beta + \beta^2 + \dots + \beta^{T-t}}{1 + \beta + \dots + \beta^{T-t}} RB_t, \quad C_t = \frac{1}{1 + \beta + \dots + \beta^{T-t}} RB_t.$$

The resulting total welfare is equal

$$V_t = \log C_t + \beta V_{t+1} = (1 + \beta + \dots + \beta^{T-t}) \log B_t + \text{const}_t,$$

confirming the guess above.

Of course, not every problem can be solved using the guess-and-verify method. In some cases,

it is not even possible to solve analytically the optimization problem in a given period. However, implemented numerically, the method is extremely powerful and can be applied to a large class of models. Moreover, increasing horizon T to infinity actually simplifies the problem. This surprising result can be easily seen from the expressions above: while $V_t(B)$ depends on t when T is finite, taking the limit $T \rightarrow \infty$, results in

$$V(B) = \frac{1}{1-\beta} \log B + \text{const},$$

where const is independent of t or B_t . Thus, the **continuation value** $V(B_t)$ depends only on the level of wealth at the beginning of the corresponding period. The optimization problem in arbitrary period of an infinitely-lived agent is given by

$$\max_{B'} \left\{ \log(RB - B') + \beta \frac{1}{1-\beta} \log B' + \beta \text{const} \right\},$$

where a prime denotes the next period's variable. The optimality condition

$$\frac{1}{RB - B'} = \frac{\beta}{1-\beta} \frac{1}{B'}$$

implies

$$B_{t+1} = \beta RB_t, \quad C_t = (1-\beta)RB_t$$

and

$$V(B) = \log C_t + \beta V(B_{t+1}) = \frac{1}{1-\beta} \log B_t + \log(1-\beta)R + \frac{\beta}{1-\beta} \log \beta R + \beta \text{const}.$$

Substituting the guess for $V(B)$ on the left hand side of the equation, we can solve for the constant term:

$$\text{const} = \frac{1}{1-\beta} \left[\log(1-\beta)R + \frac{\beta}{1-\beta} \log \beta R \right].$$

The theory of **dynamic programming** generalizes these insights. Instead of focusing on the optimal *sequences* $\{C_t, B_{t+1}\}$, it looks for time-invariant **value function** $V(\cdot)$ and **policy function** $B_{t+1} = g(\cdot)$ that solve the **Bellman equation**:

$$V(B) = \max_{B'} \left\{ u(RB - B') + \beta V(B') \right\}.$$

Intuitively, the policy function allows to recover the optimal path as $B_1 = g(B_0)$, $B_2 = g(B_1) = g(g(B_0))$, etc. To find the policy function, we need to solve the optimization problem in some period t . The state variable summarizes the past, i.e. the wealth accumulated in previous periods, while the value function summarizes the future, i.e. the maximum utility attainable in the next periods. Knowing these two objects is sufficient to resolve the trade-off

between current and future consumption.

Optimality conditions Although this is hardly the main application of the Bellman equation, the recursive methods can be used to derive the same optimality conditions that we previously obtained following the sequential approach. Indeed, the first order condition for the Bellman equation is

$$u'(RB - B') = \beta V'(B') \quad \Rightarrow \quad u'(C) = \beta V'(B'),$$

while the envelope condition for the value function, also called the **Benveniste-Scheinkman** equation, implies

$$V'(B) = u'(RB - B')R \quad \Rightarrow \quad V'(B) = u'(C)R.$$

Combining together these conditions, we get the Euler equation from the previous lecture

$$u'(C) = \beta R u'(C').$$

Computation How can we solve the Bellman equation when simple analytical expressions for $V(\cdot)$ do not exist? Luckily, the contraction mapping theorem can be used to show that under some regularity conditions, the Bellman equation has unique solution and iterating the equation starting from an arbitrary guess for a value function will eventually lead to the right solution. This formalizes the idea that $V_T(\cdot) \rightarrow V(\cdot)$ as the horizon goes to infinity $T \rightarrow \infty$. This theoretical result suggests the following numerical algorithm:

1. Start with an arbitrary function $V_0(\cdot)$.
2. For every value of B , solve the Bellman equation to obtain $V_1(\cdot)$

$$V_1(B) = \max_{B'} \left\{ u(RB - B') + \beta V_0(B') \right\}.$$

3. Iterate this procedure updating the value function at each step

$$V_{j+1}(B) = \max_{B'} \left\{ u(RB - B') + \beta V_j(B') \right\}.$$

4. Stop when the value function converges, i.e. the functions $V_{j+1}(\cdot)$ and $V_j(\cdot)$ are “close” for any values of B .
5. Solve for the policy function $B' = g(B)$.

This algorithm works surprisingly well for basic macroeconomic models and can be improved in many ways to accommodate more complex problems. The important advantage of the

dynamic programming over the sequential approach is that it can solve discrete problems. Indeed, the computation does not involve taking the derivatives and does not require the functions to be differentiable. Similarly, the method can easily accommodate occasionally binding constraints, which give rise to additional complementary slackness conditions under the sequential approach.

Theory We briefly discuss the main theoretical result about dynamic programming. Define the value function as a solution to a general dynamic problem

$$\begin{aligned} V(B_t) = \max_{\{B_{t+j+1}\}_{j=0}^{\infty}} \sum_{j=0}^{\infty} \beta^j u(B_{t+j}, B_{t+j+1}) \\ \text{s.t. } B_{t+j+1} \in \Gamma(B_{t+j}), \end{aligned} \quad (1)$$

where B is a vector, $\Gamma(\cdot)$ is a non-empty correspondence, and B_t is given. The next result shows that this problem can be restated in terms of a *functional* equation, i.e. an equation with unknown function, rather than a variable.

Theorem 1 (Principle of optimality) *Solving sequential problem is equivalent to solving the Bellman equation*

$$V(B) = \max_{B' \in \Gamma(B)} \left\{ u(B, B') + \beta V(B') \right\}. \quad (2)$$

Proof: Partition the sum into two terms:

$$\begin{aligned} V(B_t) &= \max_{\{B_{t+j+1}\}_{j=0}^{\infty}} \left\{ u(B_t, B_{t+1}) + \sum_{j=1}^{\infty} \beta^j u(B_{t+j}, B_{t+j+1}) \right\} \\ &= \max_{B_{t+1}} \left\{ u(B_t, B_{t+1}) + \max_{\{B_{t+j+1}\}_{j=1}^{\infty}} \sum_{j=1}^{\infty} \beta^j u(B_{t+j}, B_{t+j+1}) \right\} \\ &= \max_{B_{t+1}} \left\{ u(B_t, B_{t+1}) + \max_{\{B_{(t+1)+j+1}\}_{j=0}^{\infty}} \sum_{j=0}^{\infty} \beta^{j+1} u(B_{(t+1)+j}, B_{(t+1)+j+1}) \right\} \\ &= \max_{B_{t+1}} \left\{ u(B_t, B_{t+1}) + \beta V(B_{t+1}) \right\} \end{aligned}$$

where the last step follows from the definition of $V(B_{t+1})$ and the constraint $B_{t+j+1} \in \Gamma(B_{t+j})$ is suppressed to simplify notation. ■

HJB equation Just like the sequential approach, the recursive methods can be also applied to solve continuous-time problems. To see this, consider again the deterministic savings problem with zero income $Y_t = 0$ and assume an arbitrary period length Δ . Note that the value function is a stock as it reflects the continuation value *at* a given point in time. Given that $\beta \equiv e^{-\rho\Delta}$,

the Bellman equation (2) becomes

$$V(B_t) = \max_{C_t} \left\{ u(C_t)\Delta + e^{-\rho\Delta}V(B_{t+\Delta}) \right\}.$$

Because $V(B_t)$ does not affect the optimal choice, we can subtract it from both sides of the equation to obtain

$$\max_{C_t} \left\{ u(C_t)\Delta + e^{-\rho\Delta}V(B_{t+\Delta}) - V(B_t) \right\} = 0.$$

Use the approximation $e^{-\rho\Delta} \approx 1 - \rho\Delta$ for small values of Δ and rewrite the problem in terms of the control variable:

$$\max_{C_t} \left\{ u(C_t)\Delta - \rho V(B_{t+\Delta})\Delta + V(B_{t+\Delta}) - V(B_t) \right\} = 0.$$

Divide the equation by Δ and express it as follows:

$$\max_{C_t} \left\{ u(C_t) - \rho V(B_{t+\Delta}) + \frac{V(B_{t+\Delta}) - V(B_t)}{B_{t+\Delta} - B_t} \frac{B_{t+\Delta} - B_t}{\Delta} \right\} = 0.$$

Note that as $\Delta \rightarrow 0$, $B_{t+\Delta} \rightarrow B_t$ and hence, $\frac{V(B_{t+\Delta}) - V(B_t)}{B_{t+\Delta} - B_t} \rightarrow V'(B_t)$ and $\frac{B_{t+\Delta} - B_t}{\Delta} \rightarrow \dot{B}_t$:

$$\max_{C_t} \left\{ u(C_t) - \rho V(B_t) + V'(B_t)\dot{B}_t \right\} = 0.$$

Lastly, substitute in the continuous-time budget constraint from the previous lecture, drop time subscripts, and rearrange terms to get the **Hamilton-Jacobi-Bellman (HJB)** equation:

$$\rho V(B) = \max_C \left\{ u(C) + V'(B)(rB - C) \right\}.$$

There are well-developed numerical tools to solve such differential equations.

As the Bellman equation in discrete time, the HJB equation can be used to derive the Euler equation. The first order optimality condition requires

$$u'(C_t) = V'(B_t)$$

in every period of time. Therefore, differentiating this equation with respect to t , we get

$$u''(C_t)\dot{C}_t = V''(B_t)\dot{B}_t.$$

On the other hand, the envelope condition implies

$$\rho V'(B_t) = rV'(B_t) + V''(B_t)\dot{B}_t.$$

Combining these expressions, we get

$$\rho u'(C_t) = ru'(C_t) + u''(C_t)\dot{C}_t$$

which can be rewritten as the Euler equation from the previous lecture:

$$-\frac{u''(C_t)}{u'(C_t)}\dot{C}_t = r - \rho.$$

Example Consider again the case of log utility $u(C) = \log C$ and no labor income $Y_t = 0$, so that the HJB equation is given by

$$\rho V(B) = \max_C \left\{ \log C + V'(B)(rB - C) \right\}.$$

Following our results for the discrete-time problem, conjecture that $V(B) = \alpha + \gamma \log B$, where α and γ are some unknown constants. Substitute this conjecture into the HJB, take the FOC, and express consumption function as $C = \frac{B}{\gamma}$. Substitute this result together with the conjectured value function into the HJB:

$$\rho\alpha + \rho\gamma \log B = \log \frac{B}{\gamma} + \frac{\gamma}{B} \left(rB - \frac{B}{\gamma} \right).$$

Combining constant terms and terms with $\log B$, we get

$$\gamma = \frac{1}{\rho}, \quad \alpha = \frac{\log \rho + r/\rho - 1}{\rho}.$$

Finally, notice that this solution can also be obtained by writing the discrete-time value function from above for an arbitrary period length Δ and taking the limit $\Delta \rightarrow 0$.

Reinforcement learning is the cornerstone of modern AI and is gradually making its way into modern economics. It is largely based on dynamic programming, but can be applied in situations where the latter does not work. This primarily includes settings where some primitives of the model (payoffs, transition probabilities, etc.) are unknown or the state space is too large to iterate the Bellman equation (e.g. in a chess game). In both cases, one needs to explore states in an efficient way. This is what reinforcement learning (RL) does.¹

The basic idea can be illustrated using a discrete version of the pie-eating problem from above, assuming that the dimension of $\{B^1, \dots, B^N\}$ is too large to do Bellman iteration for

¹The other cornerstone of modern AI is the neural network that helps to approximate value functions in a large state space, but cannot alone solve either of the problems mentioned above.

every state:

$$V_{n+1}(B^i) = \max_j \{u(RB^i - B^j) + \beta V_n(B^j)\}, \quad i = 1, \dots, N.$$

Instead, define a state-action value function that gives continuation value for every combination of initial position and chosen savings:

$$Q(B^i, B^j) = u(RB^i - B^j) + \beta V(B^j), \quad \text{where} \quad V(B^i) = \max_j Q(B^i, B^j)$$

is still an ordinary (state) value function. The RL algorithm is then:

1. Start with an arbitrary guess for $Q_0(\cdot, \cdot)$.
2. After n iterations, suppose the algorithm visits a new state-action pair (B^i, B^j) . Update the state-action function for this combination:

$$Q_{n+1}(B^i, B^j) = u(RB^i - B^j) + \beta \max_k Q_n(B^j, B^k),$$

while leaving other values unchanged. If there are multiple new state-action pairs, update for all of them.

3. Update the corresponding value function:

$$V_{n+1}(B^i) = \max_k Q_{n+1}(B^i, B^k).$$

4. For each visited state B^i , choose the next action B^j using the current estimate of the state-action function:

$$B^j = \begin{cases} \arg \max_k Q_{n+1}(B^i, B^k), & \text{w/p } 1 - \varepsilon, \\ \text{a random feasible action,} & \text{w/p } \varepsilon. \end{cases}$$

Use newly generated state-action pairs to update $Q(\cdot, \cdot)$ by going back to step 2.

5. Continue until the state-action function $Q(\cdot, \cdot)$ converges and then recover the optimal policy function from

$$g(B^i) = \arg \max_j Q(B^i, B^j).$$

As in dynamic programming, the algorithm simultaneously picks actions and updates value functions. The main difference is that the former solves for the optimal policy everywhere: at every iteration, for each of the N states B^i , it tries all N possible savings B^j , even though most of them are likely to be irrelevant for each given state. Furthermore, it can only be applied

to all states simultaneously, which is fine if we are interested in policy functions, but might be unnecessary if the goal is to get the optimal path for a given value of B_0 . In contrast, the RL algorithm endogenously chooses which parts of the state-action space deserve attention and can be computationally much more efficient. The second fundamental difference is that dynamic programming can only be used when $u(\cdot)$ is known, while it is sufficient to learn $u(B^i, B^j)$ only for visited state-action pairs in the RL algorithm.